Lab 7: Multimodal Pipeline: Image Captioning, Translation, and Text-to-Speech

Objective

The objective of this lab is to build a multimodal AI pipeline that converts an image into spoken Nepali description. Students will use a pretrained vision-language model to generate an English caption for an image, translate that caption into Nepali, convert the translated text into speech, and finally combine all three stages into a single interactive Streamlit application.

Problem Statement

Many real-world accessibility and localization applications require converting visual content into audio in a local language. For example, describing a photo aloud to a visually impaired user in their native language. This requires chaining together three distinct AI capabilities:

- Image Captioning: using a pretrained vision-language model (BLIP) to generate a natural-language English description of an image.
- Machine Translation: translating the generated English caption into another language (Nepali, in this case).
- Text-to-Speech (TTS): converting the translated text into an audible speech file.

Each stage is first implemented and tested independently, and then all three are combined into a single end-to-end Streamlit web app that accepts an uploaded (or camera-captured) image and outputs an English caption, its Nepali translation, and a playable Nepali audio clip.

Part A: Image Captioning

Step 1: Import Required Libraries

```
from transformers import BlipProcessor, BlipForConditionalGeneration
from PIL import Image
import torch
```

Step 2: Load the Pretrained BLIP Processor and Model

```
# Load the processor and model
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")
```

Step 3: Load an Image

```
# Load your image
image = Image.open("imagess.jpeg").convert('RGB')
```

Step 4: Preprocess the Image

```
# Process the image
inputs = processor(image, return_tensors="pt")
```

Step 5: Generate a Caption

```
# Generate caption
with torch.no_grad():
    output = model.generate(**inputs)
```

Step 6: Decode and Print the Caption

```
# Decode and print the caption
caption = processor.decode(output[0], skip_special_tokens=True)
print("Caption:", caption)
```

Part B: Text Translation (`translate.ipynb`)

Step 1: Translate English Text to Nepali

```
from googletrans import Translator

translator = Translator()
result = await translator.translate("How are you?", src='en', dest='ne')
print(result.text)
```

Part C: Text-to-Speech (`tts.py`)

Step 1: Convert Nepali Text to Speech and Save/Play It

```
from gtts import gTTS
import os

# Nepali text
text = "तपाईंलाई कस्तो छ?"

# Convert text to speech
tts = gTTS(text=text, lang='ne')

# Save as mp3
tts.save("nepali_speech.mp3")

# Play the audio
Audio("nepali_speech.mp3", autoplay=True)
```

Part D: Combined Pipeline as a Streamlit App (`app.py`)

```
import streamlit as st
from transformers import BlipProcessor, BlipForConditionalGeneration, logging
from PIL import Image
import torch
from gtts import gTTS
from deep_translator import GoogleTranslator
from io import BytesIO
import threading
# Suppress transformers warnings
logging.set_verbosity_error()
# Lock for thread-safe model use
model_lock = threading.Lock()
@st.cache_resource
def load_model():
    processor = BlipProcessor.from_pretrained(
        "Salesforce/blip-image-captioning-base",
        local_files_only=False
    )
    model = BlipForConditionalGeneration.from_pretrained(
        "Salesforce/blip-image-captioning-base",
        local_files_only=False
    )
    return processor, model

processor, model = load_model()
st.title("Image Captioning with Nepali Translation and Speech")

uploaded_file = st.file_uploader("Upload an image", type=["jpg", "jpeg", "png"])
camera_file = st.camera_input("Or capture from camera")
```

```
image_file = uploaded_file or camera_file
if image_file:
    image = Image.open(image_file).convert("RGB")
    st.image(image, caption="Selected Image", use_container_width=True)
    st.subheader("Caption:")
    # Generate caption (thread-safe)
    with model_lock:
        inputs = processor(image, return_tensors="pt")
        with torch.no_grad():
            out = model.generate(**inputs)
        caption = processor.decode(out[0], skip_special_tokens=True)

    st.write("English:", caption)
    # Translate to Nepali
    translated = GoogleTranslator(source='en', target='ne').translate(caption)
    st.write("Nepali:", translated)
     # Convert to speech (no temp file)
    tts = gTTS(text=translated, lang="ne")
    mp3_bytes = BytesIO()
    tts.write_to_fp(mp3_bytes)
    mp3_bytes.seek(0)
    st.audio(mp3_bytes.read(), format="audio/mp3")
```

```
streamlit run app.py --server.address=0.0.0.0 --server.port=8501
```

Capstone Task: Now that you've seen how a full pipeline comes together; model chosen, tested alone, then wired into the next stage, pick an idea of your own and build it end to end. Combine at least two AI capabilities (vision, NLP, speech, translation, generation, whatever fits) into one working pipeline, and wrap it in something people can actually use a Streamlit or Flask app, or even just a clean command-line tool. You don't have to reuse the models from this lab; pick whatever suits your idea.